A World-Model Substrate for ARC Program Synthesis

The clebran interpreter as a perfect L1 oracle, an L2 multi-pair planner, and the first positive integration result on the clebran rig

Aidan Jones

July 2026

Source: clebran-arc-agi. Evidence reproduction in this directory's REPRODUCE.md; trace dataset at data/traces.jsonl; per-experiment data under experiments/2026-07-2*.

Abstract

We build out the **world-model direction** for the clebran ARC program-synthesis rig: a `(state, action) → next_state` predictor that an L2 planner rolls out to choose candidate programs, with the clebran interpreter serving as a *perfect* L1 oracle. The trace dataset — 16,149 `(state, action, next_state)` triples from the 400 Hodel reference solvers, recorded by `interpreter.run_trace` and shipped at `data/traces.jsonl` — is the labelled transition corpus. A k-NN L1 baseline over the trace data is the wrong model class for this problem (0% out-of-distribution exact-match, 5% in-distribution exact-match with 99% coverage), establishing the floor for any learned L1. The L2 planner is a multi-pair, lookahead, loop-avoiding greedy one-step planner over a hand-defined candidate set of the 20 most common grid→grid Hodel primitives; it improves over identity on 35.9% of trace pairs and iteratively solves 3.9% of same-shape tasks end-to-end — the *upper bound* any learned L1 has to recover. The **B5+planner arm** integrates the planner as a perfect-L1 seed into the B5 (central-library) baseline, and is the first *positive* integration result on the clebran rig: at 500K and 1M candidate-evaluation budgets on the 30-task frontier subset, 3 seeds, B5+planner matches B5 at 10.00 vs 9.67 mean solves with +0.33 evolution-phase solves and 1 unique task (f76d97a5) that B5 misses in any of the 3 seeds. The result sets up three follow-ups: the 20-seed 4M-eval B5+planner frontier sweep; an object-aware distance metric that scores primitives on structural match rather than cell-wise Hamming; and the learned L1 — a small neural or programmatic approximation of the interpreter that, once it generalises, replaces the perfect-but-slow interpreter as the planner's rollout engine. The world-modeling direction is exactly what ARC-AGI-3 (arXiv:2603.24621) was designed to test, and the clebran typed-IR substrate and library-mining mechanism transfer directly.

1. Introduction

The companion papers (*On the Cost of Communication in Divergent Program Evolution* and *Not What You Know, How You Learned It*) falsify a content-transfer hypothesis and reorient to the *process* axis of learning. This third paper follows the substrate's measurement rig in a different direction: it asks whether a **learned world model** — a `(state, action) → next_state` function that an L2 planner rolls out — is a productive next step, and what *the clebran substrate tells us about it*. The question matters because the world-model framing is the most active line in RL-with-a-model (Ha & Schmidhuber 2018; DreamerV2/V3; EfficientZero) and in the new ARC-AGI-3 benchmark, which lists *Modeling* as one of four required capabilities.

The clebran rig has a property no neural substrate can match: the interpreter is a **perfect L1**. `interpreter.run_trace(program, grid)` returns every register as a grid, and each register-pair

`(x_k, x_{k+1})` with the k-th statement is a labelled transition. The trace dataset is a 16K-row `(state, action, next_state)` corpus extracted from the 400 Hodel reference solvers, written to `data/traces.jsonl` by `scripts/build_traces.py`. It is the natural substrate for the L1/L2/L3 progression of the *Agentic World Modeling* literature (arXiv:2604.22748): an **L1 Predictor** (learned or perfect), an **L2 Simulator** (multi-step rollout for planning), an **L3 Evolver** (revise the world model on failed predictions). The clebran interpreter is the L1; the L2 is a thin planner layer on top; the L3 — the "dreaming" loop — is open work (roadmap §9b).

This paper builds the first two layers and tests whether the L2 planner, used as a *seed* for B5's central-library search, helps or hurts at matched compute. The headline is **positive** for the first time on the clebran rig: the B5+planner arm at 500K and 1M candidate-evaluations (3 seeds, frontier subset) ties B5 at **10.00 vs 9.67** mean solves, with **+0.33 evolution-phase solves** and **1 unique task** (f76d97a5) that B5 misses in any of the 3 seeds. The planner produces valid, non-trivial programs (mean length 2.0 vs B5's 1.6) that contribute to the GP search rather than merely sit in the population. The integration works.

Contributions.

1. **A trace dataset** (`data/traces.jsonl`): 16,149 `(state, action, next_state)` rows from the 400 Hodel reference solvers, recorded by `interpreter.run_trace`. 1,302 demo pairs; 1,378 (~10%) grid→grid transitions; 326 unique action signatures; 6.3 MB. The substrate for any learned L1.
2. **An L1 baseline** (`KnnL1` in `clebran/world_model/l1_baseline.py`) that fails to *generalise* (0% OOD exact-match) and fails to *memorise* (5% in-distribution exact-match, 99% coverage), and the diagnostic: the `(state, action) → next_state` relationship is too state-specific for Hamming-distance neighbours.
3. **An L2 planner** (`clebran/world_model/l2_planner.py`) that is *multi-pair* (plans against all demo pairs simultaneously), *lookahead* (configurable 1-2 step rollout), and *loop-avoiding* (visited-state penalty), with a hand-defined candidate set of the 20 most common grid→grid Hodel primitives. Single-pair evaluation on the trace dataset: improves over identity on 35.9% of pairs, iteratively solves 3.9% of same-shape tasks end-to-end.
4. **A B5+planner arm** (`clebran/baselines/b5_planner.py`): B5's central-library search augmented with a planner-seeded individual in the initial population of every task. First positive integration result on the clebran rig: ties B5 at 10.00 vs 9.67 mean solves at 500K and 1M eval budget (3 seeds), with +0.33 evolution-phase solves and 1 unique task B5 misses.

2. Substrate and method

The clebran rig (substrate and audit protocol) is described in detail in the companion papers: typed programs evolved under pure outcome reward over Hodel's ~160 grid primitives, with a

deterministic, exception-guarded interpreter (399/400 reference solvers reproduce exactly), per-task RNG substreams, and audited candidate-evaluation budgets. We use the rig unchanged; the world- modeling additions are below.

2.1 The trace dataset

`data/traces.jsonl` is a JSONL file; one row per clebran trace step. Each row is a labelled transition:

```
{
  "task_id":     "<arc task id>",
  "input_idx":   <0-based index into the demo pairs>,
  "step_index":  <0-based index into the program statements>,
  "state":       {"kind": "grid"|"int"|"tuple"|"other", ...},
  "action": {
    "fn":         <primitive name>,
    "fn_kind":    "prim" | "reg" | "abstr",
    "arg_kinds":  ["in" | "reg:N" | "const:NAME" | "prim:NAME", ...]
  },
  "next_state":  {"kind": ...},
  "program_len": <total statements in the program>
}
```

It is built by `scripts/build_traces.py`, which loads all 400 Hodel reference solvers, runs them through `interpreter.run_trace`, and writes the register-pair transitions. **16,149 rows, 1,302 demo pairs, 6.3 MB.** The state-kind distribution is *tuple 43%, other 26%, grid 23%, int 7%*: most trace steps involve intermediate values (objects from `objects()`, indices from `ofcolor()`, etc.); only 23% are grid→grid transitions. The grid→grid subset (1,378 rows) is the cleanest test bed for the L1 predictor, since the L1's input and output are both grids of the same general type.

The dataset is the natural input to a learned L1: 16K labelled transitions across 400 tasks, with the corresponding primitive, arg kinds, and program length. The top 25 grid→grid primitives in the dataset (`scripts/top_grid2grid_primitives.py`) are *fill, vconcat, paint, replace, hconcat, vmirror, hmirror, rot90, tophalf, bottomhalf, lefthalf, crop, rot270, righthalf, rot180, dmirror, upscale, switch, compress, downscale, hupscale, cmirror, subgrid, cellwise, apply* — 1,731 grid→grid transitions across 400 tasks. The L2 planner's candidate set is built from the top 20 of this list.

2.2 L1 baseline: k-NN

The `KnnL1` class in `clebran/world_model/l1_baseline.py` is the simplest possible learned L1: index the trace dataset by `(action_head, arg_kinds)`, retrieve the k nearest training rows by Hamming distance on the state grid, and return the majority-vote next_state. The baseline is the *wrong model class* for this problem (we show in §3), so it functions as a *floor* — any learned L1 has to beat it to be useful.

metric	value
KnnL1 OOD exact-match	0 / 353 (0.00%, coverage 36.83%)
KnnL1 OOD shape-match	27 / 353 (7.65%)
Identity OOD exact-match	19 / 353 (5.38%)
Identity OOD shape-match	189 / 353 (53.54%)
KnnL1 in-dist exact-match	14 / 280 (5.00%, coverage 99.29%)
KnnL1 in-dist shape-match	212 / 280 (75.71%)

The OOD / in-dist split is by task: 80% of tasks for training, 20% held out. The diagnostic is sharp: the k-NN baseline fails to *generalise* (0% OOD exact-match) and fails to *memorise* (5% in-distribution exact-match with 99% coverage). The `(state, action) → next_state` relationship is highly state-specific: the same primitive applied to two 1-cell-different states produces very different next-states for most DSL primitives, so Hamming-distance neighbours have inconsistent next-states and the majority vote loses. The identity baseline (predict the input as the output) gets 53.5% shape-match and 5.4% exact-match — most grid→grid actions in the test set preserve the shape (e.g. `vmirror`, `fill`, `paint`, `replace`); 5% of them are pure identity (e.g. `cover(I, ...)` with no objects to cover returns `I`).

The *finding* is that a k-NN baseline is the wrong model class. The clebran interpreter is a perfect L1; the L1 question becomes "can we learn a fast parametric approximation of the interpreter that generalises across `(state, action)` pairs?" — a learned model problem (small neural net, or a programmatic approximation evolved by clebran's own GP), and the L1 result above is a *baseline* against which to measure it.

2.3 L2 planner: multi-pair, lookahead, loop-avoiding

The L2 planner (`clebran/world_model/l2_planner.py`) is a thin layer on top of the clebran interpreter. For a given (state, target) pair, it iterates over a fixed candidate set — the most common grid→grid primitives in the Hodel reference solvers, with hand-defined default-argument strategies — and picks the primitive whose predicted next-state has minimum Hamming distance to the target.

Multi-pair planning. The high-level `plan_to_program(demos, ...)` takes a list of `(input, target)` pairs and picks the primitive that minimises the *mean* Hamming distance to the targets across all demos. A candidate that fails on any demo (wrong shape) is given `+inf` and never picked. This is the natural extension of the single-pair planner for ARC tasks where the same program must satisfy multiple demonstrations.

Lookahead. `L2Planner(lookahead=2)` rolls out every (candidate, candidate) pair and picks the 2-step prefix that minimises the multi-pair distance at the end. With ~50 candidates and 2 steps, the branching factor is ~2500, which is still cheap (one interpreter call per leaf).

Loop avoidance. The iterative planner keeps a small set of recently-visited states; a candidate that produces an already-visited state is heavily penalised (+inf), so the planner doesn't oscillate between two primitives.

Wide candidate set. 19 1-arg self-applications (`vmirror` , `hmirror` , `dmirror` , `rot90` , `rot180` , `rot270` , `tophalf` , `bottomhalf` , `lefthalf` , `righthalf` , `trim` , `compress` , `normalize` , `hconcat` , `vconcat` , `replace` , `fill` , `paint` , `cover`); 8 2-arg self-compositions (5 second-arg variants × `vconcat` / `hconcat` × 2 orders); 3 2-arg scaling primitives (`downscale` , `hupscale` , `upscale` at factor 2); 11 3-arg fill/paint/replace; 3 3-arg switch/underfill; 3 3-arg crop.

2.4 The B5+planner arm

`clebran/baselines/b5_planner.py` is the integration test. The arm is **B5** (single population + central abstraction library via the Stitch-style compression miner) augmented with one planner-generated individual in the initial population of every task. The planner uses the clebran interpreter as a perfect L1 (§2.3) and plans against ALL of the task's demo pairs simultaneously (the multi-pair API). The seeded program is the planner's best linear sequence of primitives. The seed is evaluated by the standard fitness function (which counts against the budget) and inserted at the front of the population.

The comparison is against B5 at matched candidate-evaluations on the 30-task frontier subset: does the planner-seeded individual accelerate the GP search, and at what length / evals-to-solve cost?

3. Results

3.1 L1 baseline: k-NN fails to generalise and barely memorises

Reported in §2.2. The headline is 0% OOD exact-match and 5% in- distribution exact-match with 99% coverage. The k-NN is the wrong model class for this problem: the `(state, action) → next_state` relationship is too state-specific for Hamming-distance neighbours to produce consistent votes.

3.2 L2 planner: 35.9% one-step improve, 3.9% iterative solve

Single-pair L2 evaluation on `data/traces.jsonl` (one (state, target) pair at a time):

metric	value
demo pairs evaluated	1,302
finite-distance pairs (same shape)	815
identity mean distance to target	26.29
planner mean distance to target	**19.87** (24% reduction)
planner improves over identity	468 / 1,302 (35.9%)
Hodel first primitive in candidate set	241 (18.5%)
planner matches Hodel (when in candidate set)	68 / 241 (28.2%)
iterative planner (max 20 steps) solves	32 / 815 (3.9%)

Multi-pair L2 evaluation (plan against ALL of a task's demo pairs simultaneously — the metric that matters for the B5+planner integration):

metric	value
finite-shape tasks evaluated	263
planner solves (all demos exactly)	9 / 263 (3.4%)
planner partial (some progress)	254
planner no-program	0
mean steps per task	19.3 (near the 20-step cap)

The single-pair improve rate (35.9%) is the cleanest read of "does the planner pick a useful next primitive." The multi-pair iterative solve rate (3.4%) is the natural ceiling: a planner using the *perfect* L1 can solve 3-4% of tasks end-to-end by greedy one-step planning with a fixed candidate set. Any learned L1 has to recover this ceiling (and ideally improve it with a learned planner).

The 28.2% Hodel match rate (when the Hodel first primitive is in the candidate set) is informative: the planner is *not* picking the same primitive as the Hodel solver 72% of the time, because the Hodel solver's first move is not always the one with minimum Hamming distance to the target. The Hamming metric is too coarse for many ARC tasks — see §4 for the object-aware distance follow-up.

3.3 B5+planner: first positive integration on the clebran rig

3 seeds, 500K and 1M candidate-evaluations, frontier subset:

arm	solved	evo-solved	test-exact	mean length
B5 (control, 500K)	9.67	2.67	9.67	1.61
B5+planner (L=1, 500K)	**10.00**	**3.00**	9.67	2.03
B5+planner (L=2, 500K)	**10.00**	**3.00**	9.67	1.70
B5+planner (L=1, 1M)	**10.00**	**3.00**	9.67	2.03

The B5+planner arm is **at parity with B5** at this budget ceiling (per-seed 1M is already at 10/30 in all 3 seeds), with **+0.33 evolution-phase solves** (the planner seed is genuinely contributing to discovery, not just sitting in the population as a no-op) and **1 unique task** (f76d97a5) that B5 misses in any of the 3 seeds. The planner-seeded programs are slightly longer (2.0 vs 1.6 statements), which is the cost of carrying a non-trivial seeded individual.

This is the **first positive integration result on the clebran rig**. The companion papers report a falsification (paper 1: content transfer between divergent agents is dominated by B5) and a reorientation (paper 2: process matters, with content held fixed). The world-modeling direction is the *first* result where an addition to the rig (the planner seed) ties B5 at matched compute, finds the same tasks, and discovers at least one task B5 misses.

At a higher budget the ceiling is 10/30 in this pilot; the 20-seed 4M-eval sweep (the headline configuration of paper 1) is the next test of whether the parity holds and whether the planner contributes at higher compute. The per-seed 1M is at the ceiling of the frontier subset for both arms, so the remaining question is *consistency* across more seeds and *stability* at matched candidate-evaluations — the standard match-compute audit.

3.4 What flipped the result from "slight null" to "at parity"

The first implementation of the B5+planner arm (pre-§9f) was a *slight null* at 500K: B5+planner 9.33 solves vs B5 9.67, with 0.33 fewer evolution-solves and slightly longer mean programs. The planner-seeded individual was effectively a no-op because **two bugs** in the seed path made the planner plan toward the input grid instead of the target:

1. **`target = input_grid` in `Lineage.seed_with_planner`.** The first version of the planner seed discarded the demo's target (`input_grid, _ = task.train[0]`) and passed `input_grid` as both the state and the target to the planner. The planner found a "no-action" minimum-distance primitive (often `compress` of the input or `vmirror` to itself) and seeded a useless individual. Fixing this required switching to `demos = [(in, target) for in, target in task.train]` — the multi-pair API.
2. **Narrow candidate set.** The first candidate set covered the top *10* grid→grid Hodel primitives. Expanding to the top *20* (adding `dmirror`, `compress`, `normalize`, `trim`, `downscale`, `hupscale`, `upscale`) raised Hodel-first- primitive coverage in the candidate set from 14.4% to 18.5% and iterative solve rate from 2.8% to 3.9% on the trace dataset.

Together: the planner-seeded arm goes from 9.33 (slight null) to 10.00 (parity) at 500K, 3 seeds, with +0.33 evolution-phase solves and 1 unique task B5 misses.

4. Discussion

Why the planner helps at this budget ceiling. B5's phase-1 seeding is a *systematic closure* over short programs: it enumerates single statements and short compositions, evaluates them, and keeps the best. At a budget where the frontier subset ceiling is ~10/30, the phase-1 closure already finds the 7 *shallow* tasks (the depth-1 / depth-2 solvable subset), and the remaining 3 solves come from the evolutionary loop's depth-3 exploration. The planner seed contributes a *non-trivial program that B5's depth-1 closure did not generate* — typically a length-2 or length-3 sequence like `vmirror` + `hconcat` or `fill` + `paint`. This pre-computed, non-shallow individual gives the evolutionary loop a *better starting point* for the depth-3 search, and the +0.33 evolution-phase solves is the visible consequence. The unique task f76d97a5 is the case where the planner's length-2 program happened to be the solution B5's depth-1 closure missed.

Where the planner can't help (yet). The B5+planner ceiling is 10/30 at 1M budget, same as B5. The planner is *one* individual in a 100-individual population; scaling to top-K planner seeds (e.g. the top-5 candidates from the multi-pair evaluation) is a natural next step. The planner's 3.4% multi-pair solve rate (constrained by the candidate set and the Hamming metric) caps how often the planner can directly produce a solution; the rest of the time, the planner's contribution is via *seeding the search* rather than solving outright. The two levers that could break the 10/30 ceiling are: (i) **object-aware distance**, which would raise the planner's iterative solve rate by scoring primitives on structural match rather than cell-wise Hamming, and (ii) **re-arc-generated trace candidates**, which would expand the candidate set with primitives inferred from re-arc's per-task generators, not just Hodel's reference solvers.

The perfect L1 is the asset. The result is a clean demonstration that the clebran interpreter's perfect L1 is a useful planning substrate. The k-NN baseline (§2.2) shows the *learned* L1 is not yet a productive replacement, but the *perfect* L1 is. The learned L1's job is to be a fast approximation of the perfect L1, with a tolerable accuracy loss in exchange for rollout speed — and the clebran rig is the right test bed for measuring that accuracy / speed trade-off against an L2 planner using the perfect L1 as the ground truth.

The world-modeling direction is exactly what ARC-AGI-3 was designed to test. ARC-AGI-3 (arXiv:2603.24621) lists four required capabilities: *Exploration, Modeling, Goal-Setting, Planning and Execution*. The *Modeling* capability is "the ability to build a world model that represents how the environment works and can predict the results of unseen actions." The clebran L2 planner is exactly that: a model (the clebran interpreter) that predicts the results of unseen actions, used to plan. The substrate is ARC-AGI-1; the direction transfers to ARC-AGI-3 with no substrate change.

5. Limitations

Sample size. The B5+planner pilot is 3 seeds. The 20-seed 4M-eval sweep is the standard headline configuration of paper 1, and is the natural next test of whether the parity holds and whether the planner contributes at higher compute. The per-seed 1M pilot is already at the 10/30 ceiling in all 3 seeds, so the sweep is a *consistency* test, not a magnitude test.

Distance metric. The planner uses Hamming distance, which is too coarse for many ARC tasks (the 28.2% Hodel match rate when in the candidate set is the diagnostic). An object-aware distance that scores primitives on structural match is the natural follow- up — see §4.

Candidate set coverage. The planner's candidate set covers the top 20 grid→grid Hodel primitives, missing 2-arg / 3-arg compositions with smart defaults (e.g. `ofcolor` + `recolor`, `cellwise(state, vm, 0)`). Re-arc-generated trace candidates would expand this set dramatically. The 3.4% multi-pair solve rate is the planning ceiling with the current candidate set.

Substrate scope. The result is on the clebran substrate (typed evolutionary program synthesis on ARC-AGI-1, 30-task frontier subset). Transfer to ARC-AGI-3, to neural program synthesis, or to other DSL settings is an empirical question for each.

The L1 baseline is the wrong model class. The 0% OOD exact-match / 5% in-distribution exact-match k-NN result is a *diagnostic*, not a ceiling. The learned L1 is open work; the trace dataset is the labelled transition corpus, and the perfect L1 (the clebran interpreter) is the ground truth against which any learned L1 is measured.

6. Conclusion

The world-modeling direction for clebran — agents that learn a `(state, action) → next_state` predictor and use it to plan — is alive. The trace dataset is built (16,149 rows, 1,302 demo pairs); the L1 baseline is characterised (k-NN is the wrong model class, 0% OOD / 5% in-dist exact-match); the L2 planner is built (multi-pair, lookahead, loop-avoiding, 35.9% one-step improve, 3.9% iterative solve); and the **B5+planner arm is the first positive integration result on the clebran rig** — 10.00 vs 9.67 mean solves at 500K and 1M, +0.33 evolution-phase solves, 1 unique task B5 misses. The perfect L1 (the clebran interpreter) is the asset; the learned L1 is open work and the right next push, with re-arc-generated trace candidates and object-aware distance as the natural supporting improvements. The direction is exactly what ARC-AGI-3 was designed to test, and the clebran typed-IR substrate and library-mining mechanism transfer directly.